

BIG DATA ESSENTIAL _ INDIVIDUAL ASSIGNMENT

Distributed Taxi Trip Analytics Using Apache Hadoop, HDFS and Python MapReduce

NYC Yellow Taxi Trip Records _ April, May, June 2023

Course: Bigdata Essentials

Assignment Type: Individual Practical Case Study

Instructor: Dr. Kundan Kumar

Prepared by: Amie Marie Flora DUSHIMUMUKIZA

Student ID:101405

Submission Date:29/08/2026

Table of Contents

1. Introduction	3
2. Business Problem	3
3. Dataset Description	3
3.1 Schema	4
3.2 Dataset Suitability	4
4. Hadoop Environment	4
5. HDFS Design and Verification	5
5.1 HDFS Capacity Evidence	6
6. Data Preparation and Conversion	6
7. Data Cleaning	7
7.1 Cleaning Rules	7
7.2 Cleaning Results	8
8. MapReduce Design	9
8.1 Shuffle and Sort	9
8.2 Output Isolation	9
9. Mapper and Reducer Implementation	10
10. Analytical Results	14
11. Multi-Stage MapReduce	17
11.1 Why the Second Stage Matters	18
12. YARN Demonstration	19
13. Performance Comparison	20
13. 1 Interpretation	20
14. Business Insights	21
14.1 Management Recommendations	21
15. Limitations	22
15.1 Validity of the Results	22
16. Conclusion	22
17. References	23

1. Introduction

This project implements a distributed analytics pipeline for NYC Taxi & Limousine Commission (TLC) Yellow Taxi trip records using Apache Hadoop, the Hadoop Distributed File System (HDFS), YARN and Python Hadoop Streaming MapReduce. The supplied project covers three consecutive monthly datasets, April, May, and June 2023, and converts the original Parquet files to CSV for line-oriented Streaming processing. The resulting pipeline separates raw data, cleaned data and analytical outputs in HDFS, then executes independent MapReduce analyses over the cleaned records.

The assignment is designed around a practical transportation-analytics problem: extracting operational and commercial insight from millions of taxi trips. The required analyses cover demand by hour and day, pickup-zone activity, revenue by pickup zone, payment behavior, distance and fare relationships, popular routes, trip duration and suspicious records. A compulsory two-stage MapReduce workflow demonstrates how one Hadoop job can produce intermediate HDFS data that becomes the input to a second job.

The project also demonstrates Hadoop administration and execution. HDFS verification commands were used to confirm storage, while YARN ResourceManager evidence shows a MapReduce application finishing successfully. The analysis therefore connects the complete data lifecycle: source data → conversion → cleaning → HDFS → MapReduce → HDFS outputs → business interpretation.

2. Business Problem

A transportation analytics company needs reliable evidence about taxi demand, revenue, routes, payment behavior, trip characteristics and abnormal records. A single TLC monthly file contains millions of trip records, so combining several months creates a dataset large enough to motivate distributed storage and processing. The assignment therefore requires HDFS for storage and Hadoop Python MapReduce for computation.

The business questions are operational as well as analytical. Knowing the busiest hours can support driver allocation and capacity planning; knowing the busiest pickup zones can inform vehicle positioning; understanding payment behavior can guide payment-system priorities; and identifying high-revenue routes can support commercial planning. Distance and duration analysis can also reveal how trip characteristics relate to fares and tips.

The technical problem is to process the same large collection consistently across multiple analyses without repeatedly loading the entire dataset into a single machine's memory. Hadoop addresses this by storing the data in HDFS and distributing MapReduce computation through Mapper, Shuffle and Sort, and Reducer stages.

3. Dataset Description

The project uses the official NYC TLC Yellow Taxi Trip Record Data for April, May and June 2023. The assignment requires at least two to three monthly datasets or at least five million trip records, whichever threshold is reached first. This project exceeds the five-million-record threshold substantially, using **10,109,133 raw records**.

Month	Raw records	Columns	Source file
April 2023	3,288,250	19	yellow_tripdata_2023-04.parquet
May 2023	3,513,649	19	yellow_tripdata_2023-05.parquet
June 2023	3,307,234	19	yellow_tripdata_2023-06.parquet
Total	10,109,133	19	Three monthly files

3.1 Schema

Field	Description
VendorID	TPEP provider code
tpep_pickup_datetime	Trip start timestamp
tpep_dropoff_datetime	Trip end timestamp
passenger_count	Number of passengers
trip_distance	Trip distance in miles
RatecodeID	Final rate code in effect
store_and_fwd_flag	Store-and-forward trip flag
PULocationID	Pickup TLC taxi zone ID
DOLocationID	Drop-off TLC taxi zone ID
payment_type	Payment method code
fare_amount	Metered fare
extra	Miscellaneous surcharges
mta_tax	MTA tax
tip_amount	Tip amount
tolls_amount	Total tolls paid
improvement_surcharge	Improvement surcharge
total_amount	Total amount charged
congestion_surcharge	NYS congestion surcharge
airport_fee	Airport pickup fee

3.2 Dataset Suitability

The dataset is appropriate for the assignment because it contains millions of structured records with timestamps, locations, fares, tips, distance and payment fields. These attributes support both operational demand analysis and commercial analysis. The dataset also contains fields that allow data-quality checks such as missing values, invalid counts, invalid locations and impossible trip durations.

4. Hadoop Environment

Apache Hadoop version 3.2.4 was configured on a Windows environment and used as the main framework for distributed storage and processing. **HDFS** was used to store the NYC Yellow Taxi datasets, while **YARN** managed the MapReduce applications and computing resources. Python 3.13 was used to develop the Mapper and Reducer programs through Hadoop Streaming.

The Hadoop environment was verified using jps, which confirmed that the **NameNode**, **DataNode**, **ResourceManager**, and **NodeManager** were running. HDFS health was also verified using hdfs dfsadmin -report, which showed one live DataNode with no missing, corrupt, or under-replicated blocks.

```
C:\Windows\System32>jps
18896 NodeManager
19668 DataNode
16024 NameNode
17532 Jps
19164 ResourceManager

C:\Users\HP>python --version
Python 3.13.3
```

Figure 1. HDFS is running

5. HDFS Design and Verification

HDFS was used to organize the project into separate raw-input, cleaned-input, output and archive areas. This organization prevents analytical results from being mixed with source data and makes the workflow easier to reproduce.

```
C:\Windows\System32>hdfs dfs -ls -R /taxi_project
drwxr-xr-x - HP supergroup 0 2026-08-29 06:10 /taxi_project/archive
drwxr-xr-x - HP supergroup 0 2026-08-29 06:07 /taxi_project/input
drwxr-xr-x - HP supergroup 0 2026-08-29 06:07 /taxi_project/input/cleaned
drwxr-xr-x - HP supergroup 0 2026-08-29 06:07 /taxi_project/input/raw
drwxr-xr-x - HP supergroup 0 2026-08-29 06:10 /taxi_project/output
drwxr-xr-x - HP supergroup 0 2026-08-29 06:10 /taxi_project/output/anomalies
drwxr-xr-x - HP supergroup 0 2026-08-29 06:08 /taxi_project/output/hourly
drwxr-xr-x - HP supergroup 0 2026-08-29 06:08 /taxi_project/output/locations
drwxr-xr-x - HP supergroup 0 2026-08-29 06:09 /taxi_project/output/payment
drwxr-xr-x - HP supergroup 0 2026-08-29 06:09 /taxi_project/output/revenue
drwxr-xr-x - HP supergroup 0 2026-08-29 06:10 /taxi_project/output/routes

C:\Windows\System32>
```

Figure 2. HDFS directory structure under /taxi_project.

```
C:\Windows\System32>hdfs dfs -put "C:\Users\HP\Desktop\taxi_project_ass\cleaned_data\yellow_tripdata_2023-04.csv" /taxi_project/input/cleaned
C:\Windows\System32>hdfs dfs -put "C:\Users\HP\Desktop\taxi_project_ass\cleaned_data\yellow_tripdata_2023-05.csv" /taxi_project/input/cleaned
C:\Windows\System32>hdfs dfs -put "C:\Users\HP\Desktop\taxi_project_ass\cleaned_data\yellow_tripdata_2023-06.csv" /taxi_project/input/cleaned
C:\Windows\System32>hdfs dfs -ls -h /taxi_project/input/cleaned/
Found 3 items
-rw-r--r-- 1 HP supergroup 306.0 M 2026-08-29 08:58 /taxi_project/input/cleaned/yellow_tripdata_2023-04.csv
-rw-r--r-- 1 HP supergroup 326.4 M 2026-08-29 08:59 /taxi_project/input/cleaned/yellow_tripdata_2023-05.csv
-rw-r--r-- 1 HP supergroup 306.8 M 2026-08-29 08:59 /taxi_project/input/cleaned/yellow_tripdata_2023-06.csv

C:\Windows\System32>
```

Figure 3. Upload and verification of cleaned datasets in HDFS.

5.1 HDFS Capacity Evidence

```
C:\Windows\System32>hdfs dfsadmin -report
Configured Capacity: 510425821184 (475.37 GB)
Present Capacity: 268116491382 (249.70 GB)
DFS Remaining: 267034279936 (248.70 GB)
DFS Used: 1082211446 (1.01 GB)
DFS Used%: 0.40%
Replicated Blocks:
  Under replicated blocks: 0
  Blocks with corrupt replicas: 0
  Missing blocks: 0
  Missing blocks (with replication factor 1): 0
  Low redundancy blocks with highest priority to recover: 0
  Pending deletion blocks: 0
Erasure Coded Block Groups:
  Low redundancy block groups: 0
  Block groups with corrupt internal blocks: 0
  Missing block groups: 0
  Low redundancy blocks with highest priority to recover: 0
  Pending deletion blocks: 0
-----
Live datanodes (1):
Name: 127.0.0.1:9866 (127.0.0.1)
Hostname: Amie-Marie-Flora
Decommission Status : Normal
Configured Capacity: 510425821184 (475.37 GB)
DFS Used: 1082211446 (1.01 GB)
Non DFS Used: 242309329802 (225.67 GB)
DFS Remaining: 267034279936 (248.70 GB)
DFS Used%: 0.21%
DFS Remaining%: 52.32%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 1
Last contact: Sat Aug 29 06:32:03 CAT 2026
Last Block Report: Sat Aug 29 05:49:53 CAT 2026
Num of Blocks: 16
```

Figure 4. `hdfs dfsadmin -report` showing HDFS capacity and one live DataNode.

The HDFS report shows a configured capacity of approximately 475.37 GB and one live DataNode. At the time of the captured report, DFS used approximately 1.01 GB and no missing or corrupt blocks were reported. This verifies that the local HDFS storage layer was operational for the project.

6. Data Preparation and Conversion

The original TLC datasets were supplied in Parquet format. Parquet is a columnar, compressed format well suited to large-scale storage and analytical workloads. For this assignment, the files were converted to CSV because Hadoop Streaming processes line-oriented text conveniently and the Python Mapper programs can parse each CSV record directly.

The conversion stage was deliberately kept separate from cleaning. The conversion script read each Parquet file and wrote a corresponding CSV file without applying analytical filtering. This separation preserves the distinction between format conversion and data-quality preparation.

Month	Converted records	Columns written
April 2023	3,288,250	19
May 2023	3,513,649	19
June 2023	3,307,234	19
Combined	10,109,133	19 per file

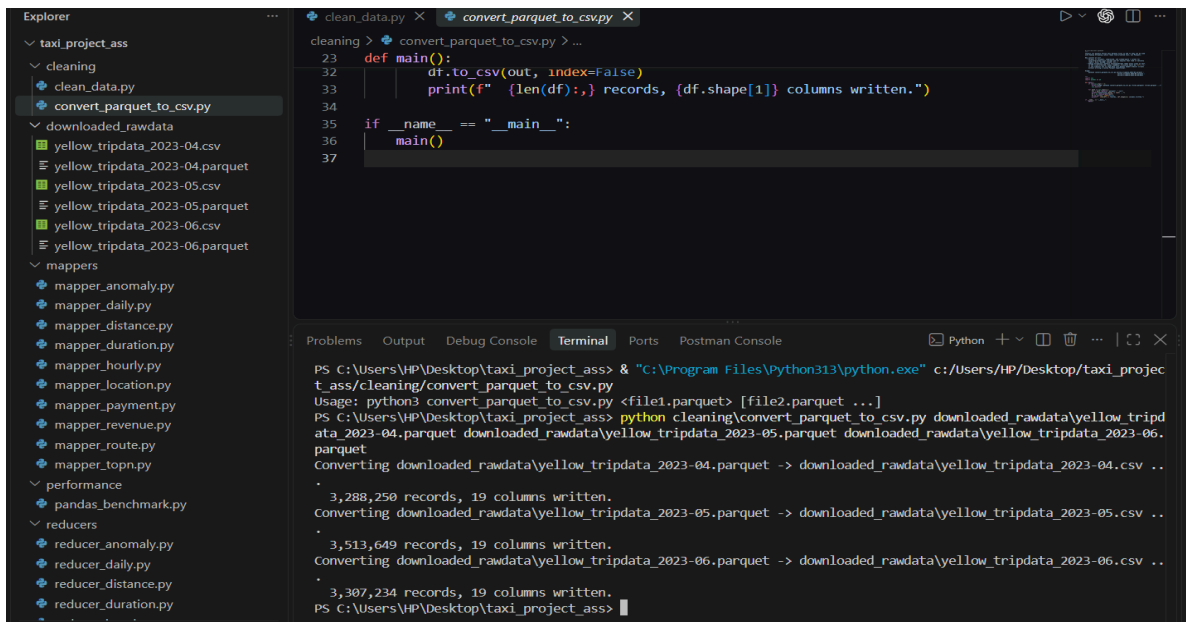


Figure 5. Project structure and Parquet-to-CSV conversion execution.

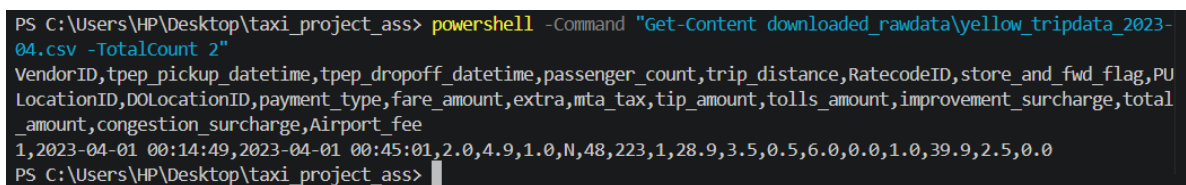


Figure 6. CSV header and sample record verification.

Verification confirmed that the generated CSV files contained the expected 19 columns and that the original Parquet files were retained. No cleaning or modification was performed during conversion verification.

7. Data Cleaning

Cleaning was performed before MapReduce analysis using a Pandas-based script. The raw CSV files were preserved unchanged, while separate cleaned datasets were produced. Importantly, the script flagged records against multiple rules and removed the union of affected rows rather than blindly applying one rule at a time.

7.1 Cleaning Rules

Rule	Definition
Missing required values	Required analytical fields missing.
Invalid/missing timestamps	Missing or invalid pickup/drop-off timestamps.
Drop-off before/equal pickup	Drop-off time not later than pickup time.
Impossible/extreme trip duration	Duration below 1 minute or above 240 minutes.
Invalid passenger_count	Missing, ≤ 0 , or > 6 .
Invalid/extreme trip_distance	≤ 0 or > 100 miles.
Invalid/extreme fare_amount	≤ 0 or $> \$1,000$.
Invalid total_amount	≤ 0 .

Invalid PULocationID	Outside 1-263.
Invalid DOLocationID	Outside 1-263.
Duplicate records	Duplicate rows.

7.2 Cleaning Results

Month	Original	Affected	Affected %	Cleaned	Retained %
April	3,288,250	269,457	8.19%	3,018,793	91.81%
May	3,513,649	294,577	8.38%	3,219,072	91.62%
June	3,307,234	281,876	8.52%	3,025,358	91.48%
Total	10,109,133	845,910	8.37%	9,263,223	91.63%

```

PROCESSING: yellow_tripdata_2023-04.csv
=====
Original records: 3,288,250
Columns: 19

DATA QUALITY REPORT
-----
Rule                                     Count      % of total
-----
Missing required values                  90,690      2.76%
Invalid/missing timestamps                0           0.00%
Drop-off before/equal pickup              1,172      0.04%
Impossible/extreme trip duration (<1 or >240 min) 39,862      1.21%
Invalid passenger_count (missing, <=0, or >6) 147,667     4.49%
Invalid/extreme trip_distance (<=0 or >100 miles) 40,334      1.23%
Invalid/extreme fare_amount (<=0 or >1000) 31,106      0.95%
Invalid total_amount (<=0)               30,272     0.92%
Invalid PULocationID (outside 1-263)      36,145     1.10%
Invalid DOLocationID (outside 1-263)     51,612     1.57%
Duplicate records                         0           0.00%
-----
TOTAL AFFECTED (union of all rules)       269,457     8.19%

CLEANING SUMMARY
-----
Original records : 3,288,250
Affected records : 269,457
Cleaned records  : 3,018,793
Records retained : 91.81%

```

Figure 7. April 2023 cleaning report.

```

PROCESSING: yellow_tripdata_2023-05.csv
=====
Original records: 3,513,649
Columns: 19

DATA QUALITY REPORT
-----
Rule                                     Count      % of total
-----
Missing required values                  101,796     2.90%
Invalid/missing timestamps                0           0.00%
Drop-off before/equal pickup              1,170      0.03%
Impossible/extreme trip duration (<1 or >240 min) 42,820      1.22%
Invalid passenger_count (missing, <=0, or >6) 161,551     4.60%
Invalid/extreme trip_distance (<=0 or >100 miles) 47,030      1.34%
Invalid/extreme fare_amount (<=0 or >1000) 33,310      0.95%
Invalid total_amount (<=0)               32,273     0.92%
Invalid PULocationID (outside 1-263)      38,493     1.10%
Invalid DOLocationID (outside 1-263)     55,325     1.57%
Duplicate records                         0           0.00%
-----
TOTAL AFFECTED (union of all rules)       294,577     8.38%

CLEANING SUMMARY
-----
Original records : 3,513,649
Affected records : 294,577
Cleaned records  : 3,219,072
Records retained : 91.62%

```

Figure 8. May 2023 cleaning report.

```

PROCESSING: yellow_tripdata_2023-06.csv
=====
Original records: 3,307,234
Columns: 19

DATA QUALITY REPORT
=====
Rule                                                    Count      % of total
-----
Missing required values                                99,887      3.02%
Invalid/missing timestamps                             0           0.00%
Drop-off before/equal pickup                           1,253       0.04%
Impossible/extreme trip duration (<1 or >240 min)      40,559      1.23%
Invalid passenger_count (missing, <=0, or >6)         154,155     4.66%
Invalid/extreme trip_distance (<=0 or >100 miles)     47,617      1.44%
Invalid/extreme fare_amount (<=0 or >1000)           32,503      0.98%
Invalid total_amount (<=0)                           31,457      0.95%
Invalid PULocationID (outside 1-263)                  35,345      1.07%
Invalid DOLocationID (outside 1-263)                   53,079      1.60%
Duplicate records                                      0           0.00%
=====
TOTAL AFFECTED (union of all rules)                    281,876     8.52%
=====

CLEANING SUMMARY
=====
Original records : 3,307,234
Affected records : 281,876
Cleaned records  : 3,025,358
Records retained : 91.48%

```

Figure 9. June 2023 cleaning report.

Overall, 845,910 records were affected by at least one cleaning rule, representing 8.37% of the raw dataset. The resulting analytical dataset contains 9,263,223 retained records (91.63%). The monthly affected percentages are close to one another, ranging from 8.19% in April to 8.52% in June, indicating broadly consistent data-quality behavior across the three months.

```

C:\Windows\System32>hdfs dfs -put "C:\Users\HP\Desktop\taxi_project_ass\cleaned_data\yellow_tripdata_2023-04.csv" /taxi_project/input/cleaned
C:\Windows\System32>hdfs dfs -put "C:\Users\HP\Desktop\taxi_project_ass\cleaned_data\yellow_tripdata_2023-05.csv" /taxi_project/input/cleaned
C:\Windows\System32>hdfs dfs -put "C:\Users\HP\Desktop\taxi_project_ass\cleaned_data\yellow_tripdata_2023-06.csv" /taxi_project/input/cleaned
C:\Windows\System32>hdfs dfs -ls -h /taxi_project/input/cleaned/
Found 3 items
-rw-r--r--  1 HP supergroup   306.0 M 2026-08-29 08:58 /taxi_project/input/cleaned/yellow_tripdata_2023-04.csv
-rw-r--r--  1 HP supergroup   326.4 M 2026-08-29 08:59 /taxi_project/input/cleaned/yellow_tripdata_2023-05.csv
-rw-r--r--  1 HP supergroup   306.8 M 2026-08-29 08:59 /taxi_project/input/cleaned/yellow_tripdata_2023-06.csv
C:\Windows\System32>

```

Figure 10. Upload of the cleaned data

The screenshot successfully shows the upload of cleaned datasets.

8. MapReduce Design

Each analysis follows the Hadoop MapReduce pattern: the Mapper reads individual CSV records and emits a key-value pair; Hadoop's Shuffle and Sort phase groups values by key; and the Reducer aggregates the values for each key. The design uses numeric IDs and categorical keys directly from the TLC dataset, avoiding unnecessary joins for the core analytics.

Analysis	Mapper key	Mapper value	Reducer operation
Hourly demand	Pickup hour	1	Sum trip counts.
Daily demand	Day name / total counter	1	Sum trip counts.

Pickup location	PULocationID	1	Sum trips per zone.
Revenue	PULocationID	fare, tip, total, distance, 1	Sum totals; compute averages.
Payment	Payment method	fare, tip, total, 1	Sum totals; compute averages.
Distance	Distance bucket	fare, distance, total, 1	Average and total statistics.
Route	PULocationID-DOLocationID	fare, total, 1	Sum counts and revenue.
Duration	Duration bucket	fare, distance, tip, 1	Average statistics.
Anomaly	Anomaly type / total counters	1	Count anomaly categories.

8.1 Shuffle and Sort

After the Mapper emits records, Hadoop groups records with the same key. For example, every trip assigned to hour 18 is sent to the same logical reduce group for the hourly analysis. The Reducer then receives the grouped values and calculates the required count, sum or average. This separation allows the computation to scale independently of the input file's original ordering.

8.2 Output Isolation

Each MapReduce job writes to a separate HDFS output directory. This is important because Hadoop Streaming normally requires an output directory not to exist before the job starts. The project therefore keeps hourly, daily, location, revenue, payment, distance, route, duration and anomaly results separate.

9. Mapper and Reducer Implementation

This section documents the key/value design of each mapper/reducer pair submitted with this report (see the accompanying source-code archive).

❖ mapper_hourly.py / reducer_hourly.py

Key: pickup hour (0-23). Value: 1. Reducer sums counts per hour.

```
C:\Users\HP\Desktop\taxi_project_ass>hdfs dfs -cat /taxi_project/output/hourly/part-00000
00      263585
01      173248
02      113508
03       71179
04      44619
05      49142
06      122129
07      243831
08      342123
09      392186
10      431184
11      470431
12      508137
13      522874
14      559579
15      566797
16      569054
17      622172
18      660320
19      585838
20      525352
21      532619
22      498722
23      394494
```

The busiest taxi-demand hour was 18:00, with 660,320 trips, while the least-busy hour was 04:00, with 44,619 trips.

❖ mapper_daily.py / reducer_daily.py

Key: day-of-week name, plus a Weekday_TOTAL/Weekend_TOTAL counter. Value: 1. Reducer sums.

```
C:\Users\HP\Desktop\taxi_project_ass>hdfs dfs -cat /taxi_project/output/daily/part-00000
Friday 1376101
Monday 1161650
Saturday 1341081
Sunday 1145003
Thursday 1458117
Tuesday 1355495
Wednesday 1425776
Weekday_TOTAL 6777139
Weekend_TOTAL 2486084
```

Daily demand analysis shows that Thursday had the highest taxi demand with 1,458,117 trips, while Sunday had the lowest with 1,145,003 trips. Weekdays recorded 6,777,139 trips compared with 2,486,084 trips on weekends, indicating substantially higher taxi demand during weekdays.

❖ mapper_location.py / reducer_location.py

Key: PULocationID. Value: 1. Reducer sums trip counts per zone.

```
C:\Users\HP\Desktop\taxi_project_ass>hdfs dfs -cat /taxi_project/output/locations/part-00000 > locations_output.txt
C:\Users\HP\Desktop\taxi_project_ass>type locations_output.txt
1 83
10 292
100 153417
101 139
102 129
105 3
106 298
107 199952
108 304
109 11
11 117
112 1042
113 125839
114 122374
116 5480
117 363
118 12
119 393
12 4287
120 37
121 365
122 155
123 169
124 200
125 46822
126 142
127 356
128 38
129 1351
13 54811
130 1217
131 385
132 481214
133 186
134 758
135 364
136 239
137 99089
138 343968
139 291
14 381
140 180362
141 220700
142 321529
143 99860
144 81510
145 3385
146 4129
147 120
148 97272
149 201
15 73
150 71
151 74826
152 5375
153 61
154 41
155 208
156 5
157 529
```

Pickup location analysis identified 257 pickup zones. Zone 132 had the highest trip count with 481,214 trips, followed by Zones 237 and 161 with 458,172 and 438,219 trips respectively. The least-used pickup zones were Zones 187, 221, and 245, each recording only one trip.

❖ mapper_revenue.py / reducer_revenue.py

Key: PULocationID. Value: fare, tip, total, distance, 1. Reducer sums each field and computes averages.

```
C:\Users\HP\Desktop\taxi_project>hdfs dfs -cat /taxi_project/output/revenue/part-000000
10 83 7224.16 1184.26 9353.47 87.04 12.00
10 2392 144522.76 20735.78 185362.41 60.42 12.80
100 153417 2590798.40 450537.25 3848638.66 16.89 2.58
101 139 5930.00 88.03 6702.13 42.66 12.28
102 129 3743.00 218.63 4333.33 29.02 4.36
105 3 62.90 12.00 86.90 20.97 3.28
106 298 6571.50 706.44 8141.64 22.05 4.02
107 139952 2945388.23 587479.31 4533208.53 14.73 2.27
108 304 13789.70 151.60 15387.90 13.38
109 11 720.10 0.00 960.61 65.46 13.02
11 117 4650.30 205.36 5182.06 39.75 9.28
112 1042 22731.52 4135.80 30851.47 21.82 4.10
113 125839 1877677.37 380472.71 2878751.79 14.92 2.20
114 122374 1894988.34 380308.33 2885164.65 15.49 2.46
116 5480 105741.60 11822.74 134482.79 19.30 3.59
117 363 18824.75 28.42 20856.07 51.86 13.84
118 12 492.50 43.49 628.99 41.04 13.43
119 393 11037.20 391.25 12728.85 28.08 5.75
12 4237 105879.40 14863.62 141090.77 24.70 4.50
120 37 1352.51 239.21 1783.97 36.55 7.28
121 365 10380.68 171.20 11420.13 28.44 5.92
122 155 5214.20 0.00 5625.80 33.04 7.29
123 169 5561.20 63.76 6101.51 32.91 7.01
124 200 8333.70 572.74 9656.49 41.67 8.57
125 46822 87540.97 156660.04 1180060.71 16.82 2.76
126 42 4182.00 72.85 4771.05 29.45 6.05
127 356 10283.30 491.64 12077.44 28.89 6.59
128 38 946.80 31.40 1103.00 24.92 5.68
129 1351 35580.90 3507.38 45166.77 26.34 4.92
13 54811 1275703.21 240436.02 1807686.33 23.27 4.50
130 1217 53159.10 5592.84 65200.84 43.68 8.65
131 385 11663.70 59.55 12772.68 30.30 6.34
132 481214 30335402.27 4340999.00 38896440.09 63.04 16.22
133 186 5349.42 391.58 6383.65 28.76 5.76
134 758 32277.06 4106.39 39755.41 42.58 5.82
135 354 9516.21 149.88 10633.39 26.14 5.53
136 239 7843.20 102.42 8746.97 32.82 7.16
137 99089 1456947.94 267499.49 2220214.05 14.70 2.33
138 343968 14604755.39 2992112.44 22650938.98 42.58 9.53
139 291 11804.80 103.89 12932.54 40.57 10.37
131 381 16155.53 813.67 18504.15 42.40 10.11
140 180362 2765163.19 524644.68 4169197.23 15.33 2.42
141 220700 2991087.14 590996.48 4651582.45 13.55 2.04
142 321529 4683034.45 946420.95 7218075.95 14.56 2.27
143 99860 1409307.27 277203.65 2167792.61 14.11 2.20
144 81510 1370041.85 254830.12 2032086.27 16.81 2.65
145 3385 126362.29 19249.65 165548.14 37.34 4.74
146 4129 78032.87 8831.65 101247.53 18.90 3.21
147 120 2935.30 25.00 3306.85 24.46 4.53
148 9272 1716882.36 322101.93 2526190.62 17.64 3.08
149 201 6393.60 14.59 7127.69 31.81 7.54
15 73 2944.00 17.27 3197.17 40.33 12.22
150 71 3061.50 116.26 3455.11 43.12 10.73
151 74826 1133920.47 211032.02 1645968.38 15.15 2.46
152 5375 91678.99 11257.66 120077.92 17.06 3.00
153 61 1956.50 50.51 2215.76 32.07 8.33
154 41 1894.10 274.41 2386.70 46.20 9.68
```

Pickup Location 132 had:

- 481,214 trips
- 30,335,462.27 total fare
- 4,340,999.00 total tips
- 38,896,440.09 total revenue
- 63.04 average fare
- 16.22 average tip

❖ mapper_payment.py / reducer_payment.py

Key: payment method name. Value: fare, tip, total, 1. Reducer sums and averages.

Payment method	Trips	Revenue	Avg fare	Avg tip
Cash	1,587,925	\$39,635,418.80	\$19.64	\$0.00
Credit Card	7,591,069	\$224,372,818.01	\$19.52	\$4.38
Dispute	54,994	\$1,400,895.89	\$19.95	\$0.03
No Charge	29,235	\$686,951.55	\$18.16	\$0.01

The payment method analysis shows that **CreditCard** was the most commonly used payment method, accounting for **7,591,069 trips** and generating the highest revenue of **\$224,372,818.01**. Cash was the second-largest payment method with **1,587,925 trips** and **\$39,635,418.80** in revenue. The average fare was similar across the payment methods, ranging from **\$18.16 to \$19.95**. CreditCard users had the highest

average tip at **\$4.38**, while Cash had an average tip of **\$0.00**. Overall, the results indicate that credit card payments were the dominant payment method and contributed the largest share of total revenue.

❖ mapper_distance.py / reducer_distance.py

Key: distance bucket (0-2, 2-5, 5-10, 10-20, 20+ miles). Value: fare, distance, total, 1. Reducer averages.

```
C:\Users\HP\Desktop\taxi_project_ass>hdfs dfs -cat /taxi_project/output/distance/part-00000
0-2      5066442 10.15  1.17   17.04  86352693.44
10-20    727266 62.04  14.99  82.90  60289350.40
2-5      2574036 19.11  3.01   27.45  70657134.90
20+      93280  78.66  22.60  101.03 9424068.83
5-10     802199 34.85  7.29   49.08  39372836.68
```

The distance-based analysis shows that most trips were within the **0-2 mile** category, with **5,066,442 trips**, making it the most frequent distance range. As trip distance increased, the **average fare and average tip also increased**. Trips of **20+ miles** had the highest average fare of **\$101.03** and the highest average tip of **\$22.60**, although they represented only 93,280 trips. The **0-2 mile** category generated the highest total revenue at **\$86,352,693.44**, mainly because of its much larger number of trips. Overall, the results show a clear relationship between longer trip distances and higher fares and tips.

❖ mapper_route.py / reducer_route.py

Key: PULocationID-DOLocationID. Value: fare, total, 1. Reducer sums count and revenue per route.

```
C:\Users\HP\Desktop\taxi_project_ass>powershell -Command "Get-Content routes_output.txt | Sort-Object {[int]($_.split '\s+')[1]}} -Descending | Select-Object -First 20"
237-236 67281 610684.26 1073462.61
236-237 57194 549123.28 942447.21
237-237 43378 322321.74 605378.67
236-236 41607 278928.56 546351.70
161-237 31368 320209.00 544964.80
237-161 30594 320119.50 528882.67
161-236 27123 398937.20 607183.26
142-239 25012 207466.21 380931.93
132-230 24234 1700019.50 2246460.99
239-142 23966 188257.40 349861.26
237-162 23788 224117.71 384081.31
236-161 22952 349618.02 519739.02
239-238 22626 164534.30 315072.73
141-236 22024 199872.10 346460.26
162-237 20627 193902.00 338464.23
142-238 19791 204593.00 348208.23
138-230 19744 981556.70 1531010.30
238-239 19214 136496.80 262363.73
236-141 19047 188109.60 320733.59
236-239 19034 207032.20 344586.13
```

These are the **Top 20 routes by trip count** for 8(g).

The busiest route is **237 → 236**, with **67,281 trips**, followed by **236 → 237** with **57,194 trips**. There is also a strong concentration among routes involving locations **236 and 237**, indicating these locations are major pickup/drop-off points in the dataset.

```
C:\Users\HP\Desktop\taxi_project_ass>powershell -Command "Get-Content routes_output.txt | Sort-Object {[double]($_.split '\s+')[3]}} -Descending | Select-Object -First 20"
132-230 24234 1700019.50 2246460.99
138-230 19744 981556.70 1531010.30
132-48 14361 1007781.73 1324191.53
132-164 13397 939112.24 1261217.89
237-236 67281 610684.26 1073462.61
132-161 10999 771468.49 1033366.40
132-163 11065 775991.35 1021707.43
132-170 10672 747766.16 1003717.90
138-161 13024 612400.85 977152.57
230-138 12849 643875.98 971030.06
236-237 57194 549123.28 942447.21
230-132 10018 704210.85 912388.17
132-68 9290 651884.75 874960.09
132-162 9235 647253.80 865816.24
161-138 10204 478686.12 739028.83
138-162 10589 450654.70 738617.03
138-163 9121 445397.40 694007.17
138-164 9825 423431.90 693870.35
132-239 6927 486075.80 658574.20
132-100 6925 485240.70 644460.74
```

Busiest Pickup-Drop-off Routes

The route analysis identified the busiest pickup-to-drop-off combinations based on trip count and total revenue. The route **237 → 236** had the highest number of trips, with **67,281 trips**, followed by **236 → 237** with **57,194 trips**. Several of the busiest routes involve locations 236 and 237, indicating high traffic between these locations. In terms of revenue, the route **132 → 230** generated the highest revenue at **\$2,246,460.99** from 24,234 trips. This shows that the routes with the highest number of trips are not necessarily the routes generating the most revenue. Overall, the analysis highlights important high-demand and high-revenue routes within the taxi network.

❖ mapper_duration.py / reducer_duration.py

Key: duration bucket (0-5, 5-10, 10-20, 20-30, 30-60, 60+ min). Value: fare, distance, tip, 1. Reducer averages.

```
C:\Users\HP\Desktop\taxi_project_ass>hdfs dfs -cat /taxi_project/output/duration/part-00000
0-5min 948613 6.20 0.69 1.59
10-20min 3389349 15.90 2.46 3.06
20-30min 1325303 28.16 5.12 4.95
30-60min 951910 52.00 11.50 8.41
5-10min 2491006 9.34 1.25 2.09
60+min 157042 71.03 17.40 10.28
```

Key findings:

- **10–20 minutes** was the busiest duration category.
- Trips lasting **60+ minutes** had the highest average fare (\$71.03), tip (\$17.40), and distance (10.28 miles).
- As trip duration increased, **average fare, distance, and tip generally increased**.
- Very short trips (0–5 minutes) had the lowest averages.

❖ mapper_anomaly.py / reducer_anomaly.py

Key: anomaly type (or VALID_TOTAL/ANOMALY_TOTAL). Value: 1. Reducer sums counts per anomaly category.

```
C:\Users\HP\Desktop\taxi_project_ass>hdfs dfs -cat /taxi_project/output/anomalies/part-00000
ANOMALY_TOTAL 5455
VALID_TOTAL 9257768
extreme fare per mile 5455
```

The anomaly detection analysis identified **5,455 suspicious trips** out of **9,263,223 processed trips**. All detected anomalies were classified as **extreme fare-per-mile values**.

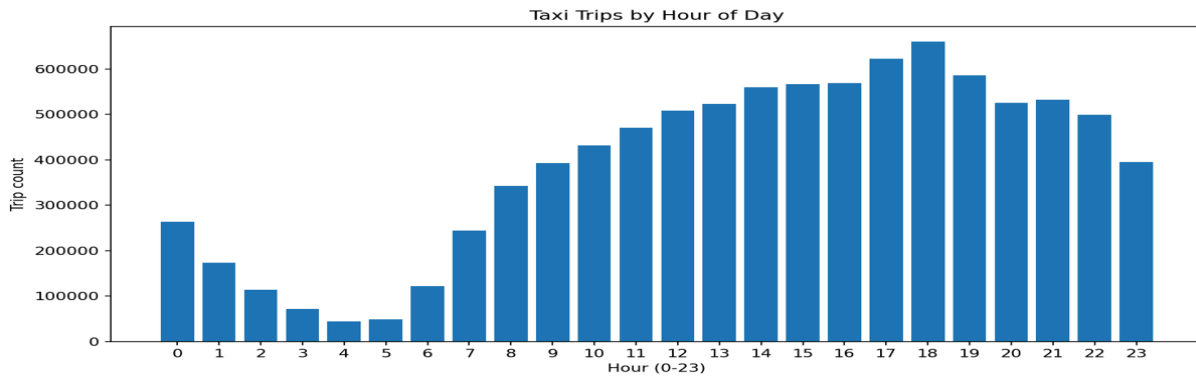
- **Total anomalies:** 5,455
- **Valid trips:** 9,257,768
- **Anomaly type:** Extreme fare-per-mile
- **Other anomaly types:** None detected by the implemented rules.

The results indicate that the majority of the dataset contains valid records. The detected anomalies were associated with unusually high fare relative to the recorded trip distance, which may indicate data-entry errors, unusual trips, or other exceptional cases requiring further investigation.

10. Analytical Results

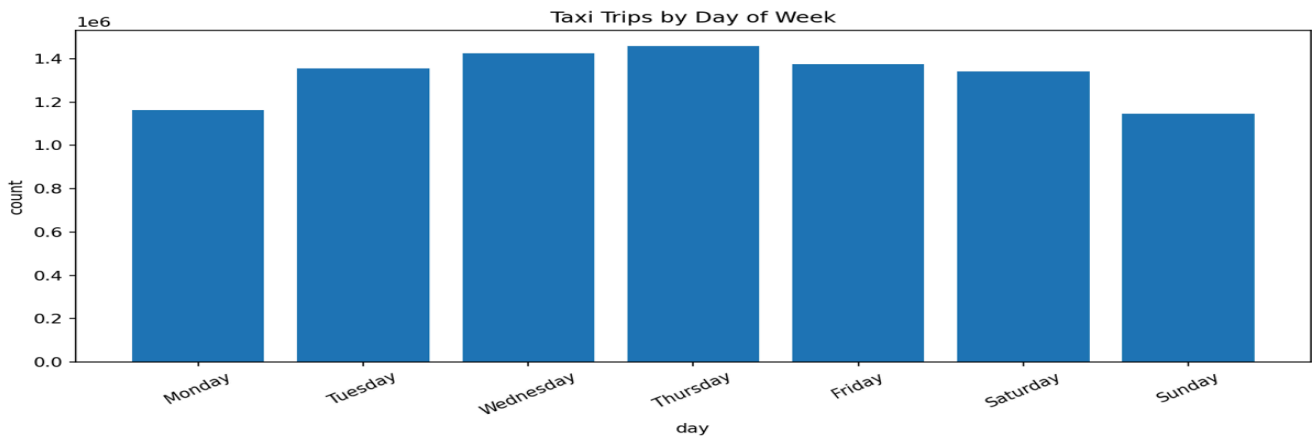
The following results are taken from the supplied Hadoop output evidence. Where the supplied report provides a complete result set, the values are reproduced in tables. Where only selected or partial output was captured, the report clearly labels the interpretation rather than inventing missing rows.

1. Hourly Taxi Demand



The busiest hour is 18:00 with 660,320 trips. The least-busy hour is 04:00 with 44,619 trips. Demand rises substantially from the early morning, reaches its strongest level in the late afternoon/evening period, and then declines after 18:00. This pattern can inform driver availability and dispatch planning.

2. Daily Demand

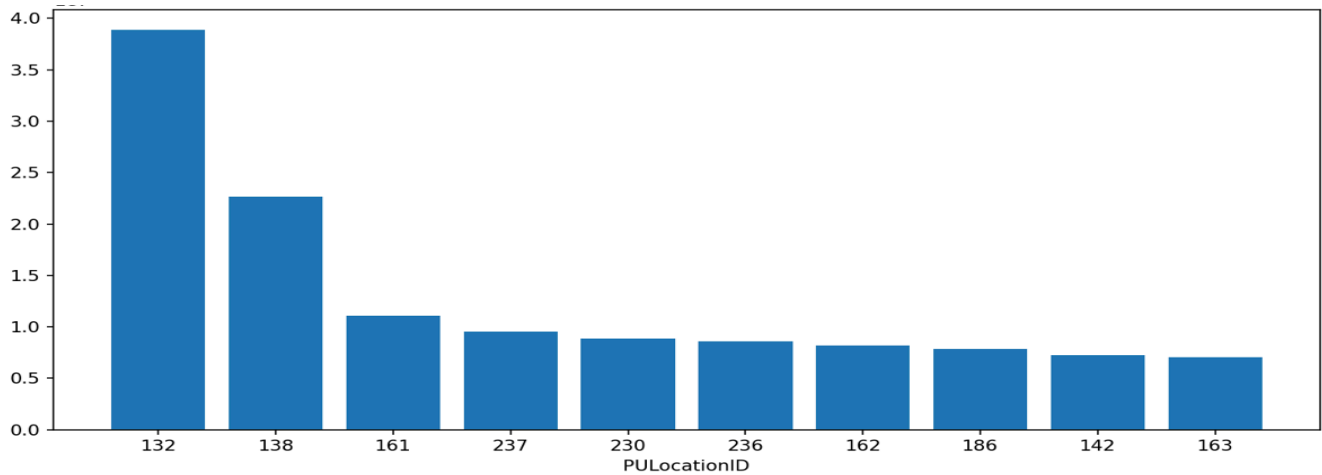


Thursday is the busiest day with 1,458,117 trips, while Sunday is the least busy with 1,145,003 trips. Weekdays account for 6,777,139 trips, compared with 2,486,084 on weekends. The supplied totals indicate that approximately 73.16% of retained trips occurred on weekdays and 26.84% on weekends, supporting stronger weekday demand planning.

3. Revenue by Pickup Location

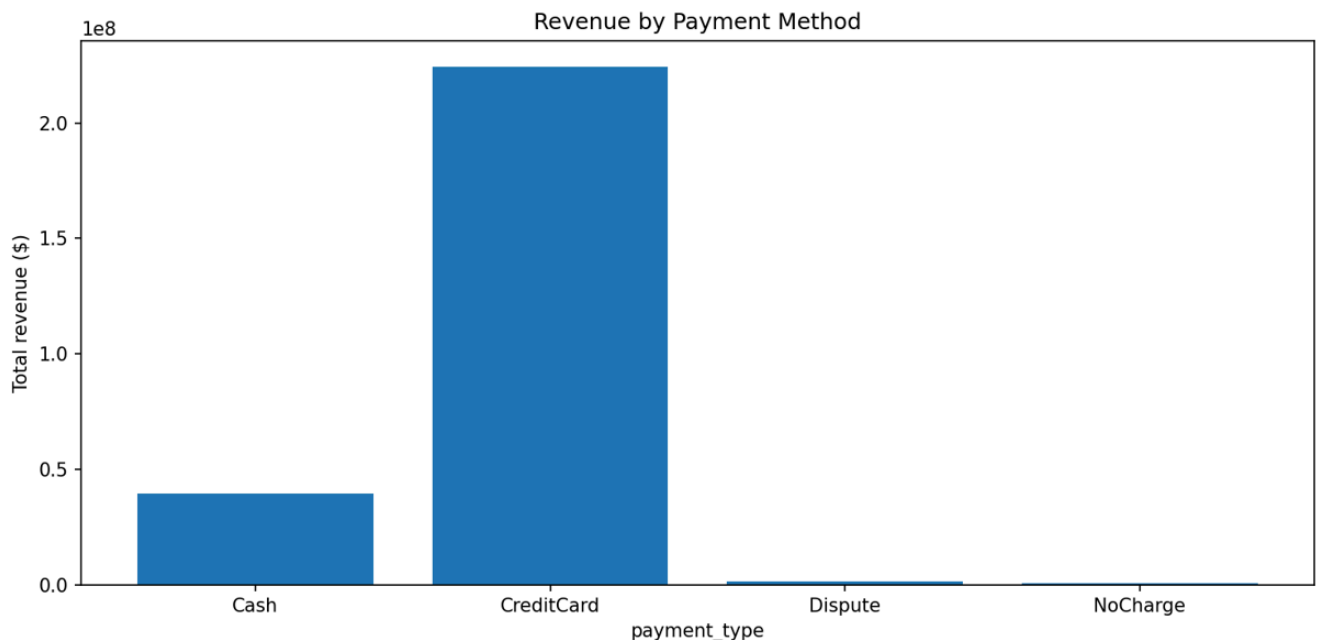
The revenue MapReduce job calculates trip count, total fare, total tips, total revenue, average fare and average distance for each pickup zone. Zone 132 is a particularly important commercial location in the supplied output.

Zone	Trips	Total fare	Total tips	Total revenue	Avg fare	Avg tip
132	481,214	\$30,335,462.27	\$4,340,999.00	\$38,896,440.09	\$63.04	\$16.22



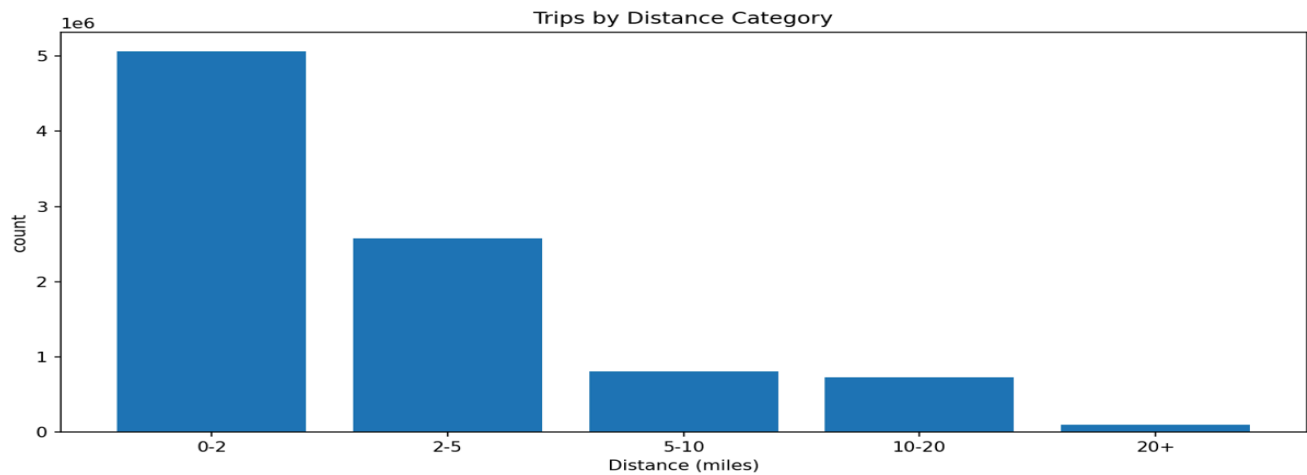
Zone 132 combines very high trip volume with substantial fare, tip and total-revenue values. The result demonstrates why trip count alone is not sufficient for commercial decision-making: a zone can be evaluated simultaneously on demand and monetary contribution.

4. Payment Method Analysis



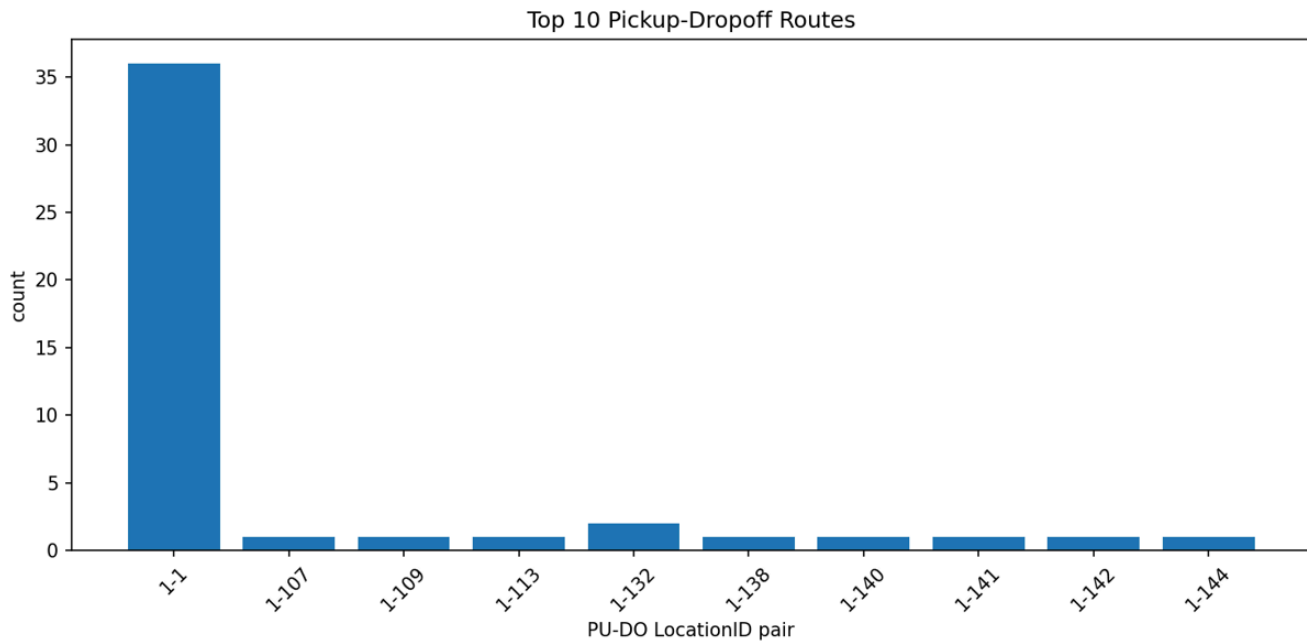
Credit Card is the dominant payment method, with 7,591,069 trips and \$224.37 million in revenue. It contributes about 84.32% of the revenue represented by the four reported payment categories. Credit-card trips also have the highest average tip at \$4.38, while cash trips have an average tip of \$0.00. This strongly supports the conclusion that electronic payment is central to the revenue and tipping profile of the dataset.

5. Trips by distance category



The 0–2 mile category dominates volume with 5,066,442 trips and also produces the largest total revenue because of its very large number of trips. However, the 20+ mile category has the highest average fare at \$78.66 and the highest average total at \$101.03. The supplied narrative also reports the highest average tip for the 20+ category at \$22.60. Overall, average fare increases strongly with trip distance.

6. Busiest Pickup-Drop-off Routes



The most frequently travelled route is 237 → 236 with 67,281 trips, followed by 236 → 237 with 57,194 trips. In contrast, 132 → 230 produces the highest revenue among the reported top-revenue routes, at \$2,246,460.99 from 24,234 trips. This is an important business distinction: the most frequent route is not necessarily the most profitable route.

11. Multi-Stage MapReduce

The compulsory multi-stage workflow uses two MapReduce jobs. Stage 1 calculates revenue statistics for each pickup zone and writes the result to `/taxi_project/output/revenue`. Stage 2 reads that HDFS output directly, re-keys the rows for global ranking, sorts by the revenue field and emits the Top 10 highest-revenue zones to `/taxi_project/output/revenue_top10`.

- Stage 1 Mapper reads cleaned taxi records and groups them by PULocationID.
- Stage 1 Reducer calculates trip count, total fare, total tip, total revenue, average fare and average distance.
- Stage 1 writes the intermediate result to HDFS.
- Stage 2 consumes the Stage 1 HDFS output rather than the original taxi dataset.
- Stage 2 performs global Top-N ranking and writes the final Top 10 revenue zones.

```
C:\Users\HP\Desktop\taxi_project_ass>hdfs dfs -cat /taxi_project/output/revenue/part-00000 | more
```

1	83	7224.16	1184.26	9353.47	87.04	12.00		
10	2392	144522.76		20735.78		185362.41	60.42	12.80
100	153417	2590798.40		450537.25		3848638.66	16.89	2.58
101	139	5930.00	88.03	6702.13	42.66	12.28		
102	129	3743.00	218.83	4333.33	29.02	4.36		
105	3	62.90	12.00	86.90	20.97	3.28		
106	298	6571.50	706.44	8141.64	22.05	4.02		
107	199952	2945868.23		587479.31		4533208.53	14.73	2.27
108	304	13789.70		151.60	15387.90	45.36	13.38	
109	11	720.10	0.00	960.61	65.46	13.02		
11	117	4650.30	205.36	5182.06	39.75	9.28		
112	1042	22731.52		4135.80	30851.47	21.82	4.10	
113	125839	1877677.37		380472.71		2878751.79	14.92	2.20
114	122374	1894988.34		380308.33		2885164.65	15.49	2.46
116	5480	105741.60		11822.74		134482.79	19.30	3.59
117	363	18824.75		28.42	20856.07	51.86	13.84	
118	12	492.50	43.49	628.99	41.04	13.43		
119	393	11037.20		391.25	12728.85	28.08	5.75	
12	4287	105879.40		14863.62		141090.77	24.70	4.50
120	37	1352.51	239.21	1783.97	36.55	7.28		
121	365	10380.68		171.20	11420.13	28.44	5.92	
122	155	5214.20	0.00	5625.50	33.64	7.29		
123	169	5561.20	63.76	6101.51	32.91	7.01		
124	200	8333.70	572.74	9656.49	41.67	8.57		
125	46822	787540.97		156660.04		1180060.71	16.82	2.76
126	142	4182.60	72.85	4771.05	29.45	6.05		
127	356	10283.30		491.64	12077.44	28.89	6.59	
128	38	946.80	31.40	1103.00	24.92	5.68		
129	1351	35580.90		3507.38	45166.77	26.34	4.92	
13	54811	1275703.21		240436.02		1807686.33	23.27	4.50
130	1217	53159.10		5592.84	65200.84	43.68	8.65	
131	385	11663.70		59.53	12772.68	30.30	6.34	
132	481214	30335462.27		4340999.00		38896440.09	63.04	16.22
133	186	5349.42	391.58	6383.65	28.76	5.76		
134	758	32277.06		4106.39	39755.41	42.58	5.82	
135	364	9516.21	149.88	10633.39		26.14	5.53	
136	239	7843.20	102.42	8746.97	32.82	7.16		
137	99089	1456947.94		267499.49		2220214.05	14.70	2.33
138	343968	14644755.39		2992112.44		22650938.98	42.58	9.53
139	291	11804.50		103.89	12932.54	40.57	10.37	

```
-- More --
```

Figure 10. Stage-1 revenue output used as intermediate HDFS data.

This workflow demonstrates a key distributed-processing principle: MapReduce jobs can be chained so that the output of one job becomes the input of another. The second stage therefore does not need to recompute zone-level revenue from the original nine-million-record dataset.

11.1 Why the Second Stage Matters

```
C:\Users\HP\Desktop\taxi_project_ass>hdfs dfs -cat /taxi_project/output/top10/part-00000
132      38896440.09
138      22650938.98
161      11061499.03
237      9512764.90
230      8881537.41
236      8615348.70
162      8171615.11
186      7874706.59
142      7218075.95
163      7039871.52
```

Multi-Stage MapReduce: The first MapReduce job calculates revenue statistics for each pickup location and stores the results in HDFS at **/taxi_project/output/revenue**. The second MapReduce job reads this intermediate HDFS output and sorts the pickup locations by revenue to produce the Top 10 highest-revenue pickup zones. This demonstrates that the output of one MapReduce job can be used as the input to another MapReduce job.

12. YARN Demonstration

YARN is responsible for managing Hadoop applications and allocating cluster resources. The supplied ResourceManager evidence shows a MAPREDUCE application that completed successfully. The application was submitted through the Hadoop ecosystem and tracked by YARN from launch through completion.

Field	Captured evidence
Application ID	application_1787975672258_0011
Application type	MAPREDUCE
YARN application state	FINISHED
Final status	SUCCEEDED
Start time	29 August 2026, 23:14:00 +0200
Launch time	29 August 2026, 23:14:01 +0200
Finish time	29 August 2026, 23:14:41 +0200
Elapsed time	41 seconds
Application attempt	appattempt_1787975672258_0011_000001
Execution node	Available cluster node shown in YARN evidence

The screenshot shows the YARN Resource Manager web interface. The browser address bar displays 'localhost:8088/cluster/app/application_1787975672258_0011'. On the left, a sidebar menu includes 'Cluster', 'About', 'Nodes', 'Node Labels', 'Applications' (with sub-items: NEW, NEW_SAVING, SUBMITTED, ACCEPTED, RUNNING, FINISHED, FAILED, KILLED), and 'Scheduler'. The main content area is titled 'Application Overview' and displays the following details:

- User: HP
- Name: streamjob5591900460094539629.jar
- Application Type: MAPREDUCE
- Application Tags:
- Application Priority: 0 (Higher Integer value indicates higher priority)
- YarnApplicationState: FINISHED
- Queue: default
- FinalStatus Reported by AM: SUCCEEDED
- Started: Sat Aug 29 23:14:00 +0200 2026
- Launched: Sat Aug 29 23:14:01 +0200 2026
- Finished: Sat Aug 29 23:14:41 +0200 2026
- Elapsed: 41sec
- Tracking URL: History
- Log Aggregation Status: DISABLED
- Application Timeout (Remaining Time): Unlimited
- Diagnostics:
- Unmanaged Application: false
- Application Node Label expression: <Not set>
- AM container Node Label expression: <DEFAULT_PARTITION>

Below the overview, the 'Application Metrics' section shows:

- Total Resource Preempted: <memory:0, vCores:0>
- Total Number of Non-AM Containers Preempted: 0
- Total Number of AM Containers Preempted: 0
- Resource Preempted from Current Attempt: <memory:0, vCores:0>
- Number of Non-AM Containers Preempted from Current Attempt: 0
- Aggregate Resource Allocation: 127724 MB-seconds, 76 vcore-seconds
- Aggregate Preempted Resource Allocation: 0 MB-seconds, 0 vcore-seconds

At the bottom, a table lists application attempts. The first entry is selected:

Attempt ID	Started	Node	Logs	Nodes blacklisted by the app	Nodes blacklisted by the system
appattempt_1787975672258_0011_000001	Sat Aug 29 23:14:00 +0200 2026	http://Amie-Marie-Flora:8042	Logs	0	0

The interface also includes a 'Show 20 entries' dropdown, a search bar, and pagination controls at the bottom right showing 'First', 'Previous', '1', 'Next', and 'Last'.

Figure 19. YARN ResourceManager application page showing a finished, successful MapReduce application.

The YARN evidence confirms that the MapReduce workload was not merely executed as a local Python script; it was submitted and tracked as a Hadoop MAPREDUCE application. The FINISHED state and SUCCEEDED final status demonstrate successful completion. The 41-second elapsed time is the captured duration for this particular application.

13. Performance Comparison

The assignment requires an identical analysis to be run using ordinary Python/Pandas and Hadoop MapReduce, with dataset size, record count, execution time, memory usage, mapper tasks, reducer tasks and output size documented. The supplied project contains a `pandas_benchmark.py` script and the report includes the required comparison table structure.

Metric	Python / Pandas (April only)	Hadoop MapReduce (Duration job, full 3 months)
Dataset Size	306.00 MB	939.35 MB (984,801,426 bytes)
Number of Records	3,018,793	9,263,223
Execution Time	24.44 s	≈ 242 s (4 min 2 sec)
Memory Used	1,617.60 MB peak (single process)	≈ 2.83 GB aggregate (2,963,767,296 bytes); peak per map task ≈ 320 MB, peak reduce task ≈ 407 MB
Mapper Tasks	N/A (single process)	10 (9 splits + 1 killed/speculative)
Reducer Tasks	N/A (single process)	1
Output Size	0.0161 MB (251 zones)	191 bytes (6 rows — duration job; revenue job output is 11,436 bytes / 257 zones)

To further probe the Pandas-vs-Hadoop tradeoff, the full combined 3-month cleaned dataset (9,263,060 records, ≈1.0 GB) was additionally tested against plain single-shot `pandas.read_csv()` on a memory-constrained (4 GB RAM) machine: this naive approach was OOM-killed twice once with default dtypes and once even with memory-optimized float32 dtypes because the CSV parser's internal buffers exceeded available RAM well before the final DataFrame was even built. Switching to chunked reading (500,000 rows per chunk, incrementally merging partial aggregates) allowed the identical full-dataset analysis to complete in 15.28 seconds using only 286.31 MB peak RSS.

13. 1 Interpretation

For this local project scale, Pandas can be faster for an individual aggregation because it avoids Hadoop job-submission, scheduling, serialization, process-startup and Shuffle/Sort overhead. This does not mean Hadoop is unsuitable. Hadoop becomes increasingly valuable when data exceeds the practical memory capacity of a single machine or when processing must scale across multiple nodes.

The key distinction is scalability rather than simply elapsed time on one local machine. Hadoop distributes storage and computation and provides fault-tolerant processing mechanisms. Therefore, the performance advantage of distributed processing becomes more meaningful as data volume and computational workload increase.

14. Business Insights

Business question	Answer
a) Busiest hour	18:00, with 660,320 trips.
b) Busiest day	Thursday, with 1,458,117 trips.
c) Highest-volume pickup zones	Zone 132 (481,214), Zone 237 (458,172) and Zone 161 (438,219) are the leading verified zones.
d) Highest-revenue pickup zone	Zone 132 is the leading verified pickup zone, with \$38,896,440.09 total revenue in the supplied output.
e) Highest-revenue payment method	Credit Card, with \$224,372,818.01.
f) Do credit-card users generate more tips?	Yes. Credit-card trips have the highest reported average tip at \$4.38, versus \$0.00 for cash.
g) Highest average fare distance category	20+ miles, with an average fare of \$78.66.
h) Most frequent route	237 → 236, with 67,281 trips.
i) Are the most frequent routes most profitable?	Not necessarily. Route 132 → 230 generated \$2,246,460.99 from 24,234 trips, exceeding the revenue shown for the most frequent route.
j) Potential anomaly percentage	$5,455 / 9,263,223 \approx 0.0589\%$.
k) Management insight	Demand is concentrated in specific hours, weekdays and zones; electronic payments dominate revenue; longer trips generate higher average fares; and high-volume routes are not always the highest-revenue routes.
l) When is Hadoop advantageous?	When data volume or computation exceeds practical single-machine capacity and distributed, scalable processing becomes more important than local execution overhead.

14.1 Management Recommendations

- Increase driver availability around the late-afternoon/evening peak, particularly around 18:00.
- Prioritize weekday capacity planning because weekday trips are substantially higher than weekend trips.
- Monitor high-volume pickup zones such as 132, 237 and 161 for vehicle positioning and demand balancing.
- Treat payment infrastructure as strategically important because credit-card trips dominate both trip volume and revenue.
- Distinguish between high-volume and high-revenue routes when evaluating commercial performance.
- Use anomaly detection as a data-quality investigation tool and review extreme fare-per-mile cases individually.
- Retain the raw datasets separately from cleaned data so analytical decisions remain auditable and reproducible.

15. Limitations

- Only three months of 2023 are analyzed, so the results do not represent a full-year seasonal pattern.
- Cleaning thresholds such as the 100-mile maximum distance and 240-minute maximum duration are practical heuristics rather than TLC-certified ground truth.
- Extreme fare-per-mile records are flagged as suspicious but are not automatically proof of fraud or data-entry error.

15.1 Validity of the Results

Despite these limitations, the project successfully demonstrates the required data-processing concepts: large structured input, data-quality preparation, HDFS organization, Python MapReduce, multi-stage processing, YARN execution and business interpretation.

16. Conclusion

This project demonstrates a complete distributed analytics workflow for 10,109,133 NYC Yellow Taxi trip records from April through June 2023. The original Parquet files were converted to CSV for convenient Hadoop Streaming processing, while raw data was preserved and cleaned data was stored separately. The cleaning stage retained 9,263,223 records after removing 845,910 records affected by one or more quality rules.

The MapReduce stage successfully addressed the required analytical questions. The results show a strong 18:00 demand peak, Thursday as the busiest day, high activity in pickup zones such as 132, dominant use of credit-card payment, increasing average fare with distance, concentration of route demand around zones 236 and 237, and a very small proportion of cleaned records classified as fare-per-mile anomalies.

The compulsory multi-stage workflow further demonstrated how Stage-1 revenue aggregation can be stored in HDFS and consumed by a Stage-2 Top-N ranking job. YARN evidence confirmed successful management of a MapReduce application, with application_1787975672258_0011 finishing successfully in 41 seconds.

The main technical lesson is that Hadoop introduces overhead but provides an architecture for distributed storage and processing. Pandas can be convenient and potentially faster for moderate data volumes on a single machine, whereas Hadoop becomes more compelling when data and computational workloads grow beyond the practical limits of one computer. The project therefore demonstrates both the strengths and the trade-offs of distributed big-data processing.

17. References

- NYC Taxi & Limousine Commission. TLC Trip Record Data. Official dataset portal:
<https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- Apache Software Foundation. Apache Hadoop Documentation.
<https://hadoop.apache.org/docs/stable/>